UTS

PEMBELAJARAN MESIN



NAMA : I Nyoman Arya Mas Sugiarthawan

NIM : 202431108

KELAS : H

DOSEN : Rizqia Cahyaningtyas, S.T., M.Kom.

NO.PC :

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II 4
LANDASAN TEORI 4
2.1 Klasifikasi Pembelajaran Mesin 4
2.2 Klasifikasi pada Dataset Counter-Strike 4
2.3 Exploratory Data Analysis (EDA) 4
2.4 Preprocessing Data 5
2.5 Algoritma Gaussian Naive Bayes 5
2.6 Algoritma K-Nearest Neighbors (KNN) 6
2.7 Evaluasi Model Klasifikasi 6
2.8 Ketidakseimbangan Kelas dan Pemilihan Fitur 6
BAB III 8
HASIL 8
3.1. Lingkungan dan Dataset 8
3.2. Penjelasan Setiap Cell Kode 8
BAB IV 29
PENUTUP 29
DAFTAR PUSTAKA 30

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana struktur dan karakteristik dataset 03_counter_strike_2.csv dapat dipahami melalui Exploratory Data Analysis (EDA)?
2. Bagaimana menyiapkan fitur numerik, boolean, dan kategorikal agar dapat diproses oleh model Gaussian Naive Bayes?
3. Bagaimana menerapkan algoritma Gaussian Naive Bayes dan K-Nearest Neighbors (KNN) untuk memprediksi tim pemenang ronde (T atau CT) berdasarkan kondisi ronde pada saat pencuplikan data?
4. Bagaimana memperoleh dan mengevaluasi tingkat akurasi kedua algoritma menggunakan data training dan data testing?
5. Bagaimana membandingkan kinerja Gaussian Naive Bayes dan KNN berdasarkan Accuracy, Precision, Recall, F1-Score, dan Confusion Matrix, serta bagaimana pengaruh asumsi independensi fitur, parameter var_smoothing, dan nilai k terhadap kinerja masing-masing?

1.2 Tujuan Masalah

1. Memahami struktur, atribut, dan tipe data pada dataset 03_counter_strike_2.csv.
2. Melakukan eksplorasi data untuk mengetahui distribusi pemenang ronde, sebaran sisa waktu, pengaruh peta dan status bom, serta korelasi antarfitur numerik.
3. Melakukan preprocessing berupa pemeriksaan missing value dan duplikasi, pemisahan fitur numerik dan kategorikal, encoding, serta standardisasi.
4. Membangun dan melatih model klasifikasi menggunakan algoritma Gaussian Naive Bayes dan KNN.
5. Menguji kemampuan kedua model dalam memprediksi pemenang ronde pada data testing.
6. Mengukur kinerja kedua model menggunakan Accuracy, Precision, Recall, F1-Score, dan Confusion Matrix.
7. Membandingkan hasil kedua algoritma untuk mengetahui kelebihan dan keterbatasan masing-masing, termasuk dampak asumsi independensi fitur pada Naive Bayes dan pemilihan nilai k pada KNN.

1.3 Manfaat Masalah

1. Memahami konsep dasar pembelajaran mesin dan penerapannya pada klasifikasi data.
2. Meningkatkan kemampuan mengolah data dengan Python dan pustaka Scikit-learn.
3. Memahami tahapan EDA, preprocessing, training, testing, dan evaluasi model secara berurutan.
4. Memahami cara kerja Gaussian Naive Bayes (Teorema Bayes dan asumsi independensi fitur) dan KNN (kedekatan jarak), serta perbedaan karakteristik keduanya pada data nyata.

BAB II
LANDASAN TEORI

2.1 Klasifikasi Pembelajaran Mesin

Pembelajaran mesin (machine learning) adalah cabang kecerdasan buatan yang membuat komputer mampu menemukan pola dari data sehingga keputusan atau prediksi dapat dihasilkan tanpa aturan yang ditulis satu per satu. Pada pembelajaran terawasi (supervised learning), model dilatih dengan data yang sudah memiliki label. Dari data tersebut model mempelajari hubungan antara fitur masukan dan label keluaran, lalu memakainya untuk menebak label pada data yang belum pernah dilihat. Klasifikasi adalah salah satu bentuk pembelajaran terawasi, yaitu ketika label yang diprediksi berupa kategori diskrit.

Praktikum ini memakai dua algoritma, yaitu Gaussian Naive Bayes yang bersifat probabilistik dan K-Nearest Neighbors (KNN) yang berbasis kedekatan data. Dani dan Ginting (2024) [1] mempelajari klasifikasi dengan Gaussian Naive Bayes dan membandingkannya dengan decision tree menggunakan ukuran recall, precision, accuracy, dan F1-score, yaitu rangkaian ukuran yang juga dipakai dalam laporan ini.

2.2 Klasifikasi pada Dataset Counter-Strike

Dataset 03_counter_strike_2.csv berisi cuplikan (snapshot) kondisi ronde pada permainan Counter-Strike, yaitu gim tembak-menembak taktis yang mempertemukan dua tim: Terrorist (T) dan Counter-Terrorist (CT). Setiap baris menggambarkan keadaan satu ronde pada satu saat, antara lain sisa waktu, skor kedua tim, peta, status bom, total health, armor, uang, dan jumlah helm. Variabel target round_winner bernilai T atau CT sehingga permasalahannya adalah klasifikasi biner, yaitu memprediksi tim yang akhirnya memenangkan ronde dari kondisi yang terlihat pada saat pencuplikan.

Memperkirakan peluang menang dari keadaan permainan merupakan topik yang banyak dikaji pada esports. Xenopoulos dkk. (2022) [3] membangun model probabilitas kemenangan yang dapat diinterpretasi pada data Counter-Strike: Global Offensive dan menemukan bahwa fitur seperti total HP tim dan nilai perlengkapan termasuk yang penting. García-Méndez dan de Arriba-Pérez (2025) [4] meneliti prediksi pemenang CS:GO yang dapat dijelaskan dalam lingkungan streaming, dengan Gaussian Naive Bayes sebagai model dasar (baseline) di samping dua model pohon. Kedua penelitian ini menunjukkan bahwa kondisi ronde memuat informasi yang berguna bagi prediksi pemenang, dan Gaussian Naive Bayes layak digunakan sebagai titik awal.

2.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) adalah tahap penjajakan data sebelum pemodelan untuk mengetahui struktur, tipe data, sebaran nilai, kejanggalan, serta hubungan antarvariabel. Tahap ini umumnya terdiri atas pemeriksaan tipe data dan statistik deskriptif, pengecekan distribusi target, pengamatan distribusi fitur, dan pengukuran korelasi. Grafik seperti countplot, histogram, dan heatmap korelasi membantu menampakkan pola yang sulit terbaca dari tabel angka.

Pada dataset ini EDA dipakai untuk melihat proporsi kemenangan T dan CT, sebaran sisa waktu ronde, perbandingan pemenang pada setiap peta dan menurut status bom, serta korelasi antarfitur numerik. Hasil korelasi penting bagi Naive Bayes karena algoritma ini mengasumsikan fitur saling bebas. Korelasi Pearson hanya mengukur hubungan linear antarvariabel numerik dan tidak dapat dipakai untuk menyimpulkan sebab-akibat.

2.4 Preprocessing Data

Preprocessing adalah tahap menyiapkan data agar sesuai dengan kebutuhan algoritma. Langkahnya meliputi pemeriksaan nilai kosong dan duplikasi, pemisahan fitur dari target, pengubahan variabel non-numerik menjadi angka, penskalaan, dan pembagian data. Dataset yang digunakan memiliki 2.000 baris dan 14 kolom. Tidak ada nilai kosong, tetapi terdapat 39 baris duplikat. Baris duplikat tidak dihapus karena cuplikan dari kondisi ronde yang sama atau mirip dapat memang bernilai identik, dan jumlahnya hanya 1,95% dari seluruh data.

Kolom map bertipe teks sehingga diubah dengan One-Hot Encoding, yaitu membuat satu kolom indikator untuk setiap kategori peta tanpa memberi urutan numerik buatan. Kolom bomb_planted yang bertipe boolean diubah menjadi 0 dan 1. Fitur numerik distandardisasi dengan StandardScaler, yang menggeser rata-rata ke nol dan menyamakan simpangan baku menjadi satu. Berbeda dengan algoritma berbasis jarak, Gaussian Naive Bayes menaksir rata-rata dan varians setiap fitur secara terpisah per kelas sehingga tidak terlalu bergantung pada skala. Standardisasi tetap berguna karena parameter penghalus var_smoothing ditentukan secara proporsional terhadap varians terbesar di antara fitur; tanpa standardisasi, fitur uang yang bervarians sangat besar akan mendominasi penghalusan tersebut.

Data dibagi 80% untuk training dan 20% untuk testing dengan stratified splitting agar proporsi kelas pada kedua bagian sama dengan data asli. Encoder dan scaler dipasang di dalam pipeline sehingga parameternya hanya dipelajari dari data training, dan informasi data testing tidak ikut terpakai saat pelatihan.

2.5 Algoritma Gaussian Naive Bayes

Naive Bayes adalah keluarga klasifikator probabilistik yang bertumpu pada Teorema Bayes. Dengan X sebagai vektor fitur dan C sebagai kelas, teorema tersebut dituliskan sebagai berikut.

$$P(C \mid X) = [\, P(X \mid C) \times P(C) \,] \,/\, P(X)$$

P(C) adalah probabilitas prior, yaitu peluang suatu kelas sebelum fitur diamati dan pada praktikum ini diestimasi dari proporsi kelas pada data training. P(X | C) adalah likelihood, yaitu peluang munculnya fitur-fitur tersebut jika kelasnya C. P(C | X) adalah probabilitas posterior, yaitu peluang kelas setelah fitur diamati. P(X) berlaku sama bagi semua kelas sehingga tidak memengaruhi perbandingan antarkelas. Kelas prediksi dipilih dari posterior terbesar.

Kata “naive” merujuk pada asumsi independensi bersyarat: setelah kelas diketahui, setiap fitur dianggap tidak bergantung pada fitur lain. Asumsi ini memecah likelihood gabungan menjadi hasil kali likelihood masing-masing fitur.

$$P(C \mid x_1, \ldots, x_n) \propto P(C) \times P(x_1 \mid C) \times P(x_2 \mid C) \times \ldots \times P(x_n \mid C)$$

Pada varian Gaussian, setiap fitur kontinu di dalam suatu kelas dianggap berdistribusi normal dengan rata-rata μ dan varians σ^2 yang ditaksir dari data training.

$$P(x_j \mid C) = 1 \,/\, \sqrt{(2\pi\sigma^2_{cj})} \times \exp\!\left(-\, (x_j - \mu_{cj})^2 \,/\, (2\sigma^2_{cj}) \right)$$

Srisuradetchai (2025) [2] menjelaskan bahwa Gaussian Naive Bayes dilandasi Teorema Bayes dengan asumsi independensi bersyarat antarfitur dan likelihood normal bagi fitur kontinu, serta mencatat bahwa asumsi independensi dapat membatasi kinerja bila fitur pada data nyata saling berkorelasi. Dani dan Ginting (2024) [1] juga menyebut bahwa asumsi independensi umumnya tidak terpenuhi sempurna, tetapi klasifikator ini tetap kompetitif. Keunggulan algoritma ini adalah

sederhana, cepat dilatih, dan menghasilkan probabilitas kelas. Kelemahannya adalah sensitif terhadap pelanggaran asumsi independensi dan distribusi Gaussian, misalnya pada fitur biner hasil One-Hot Encoding atau fitur yang miring.

Scikit-learn juga menyediakan parameter var_smoothing, yaitu porsi varians terbesar yang ditambahkan ke varians setiap fitur untuk menjaga kestabilan perhitungan. Nilai bawaannya 10^{-9}, dan pengaruhnya diuji pada Cell 21.

2.6 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors adalah algoritma klasifikasi berbasis kedekatan (instance-based learning) yang tidak membangun model eksplisit. Data training hanya disimpan; ketika ada data baru, algoritma menghitung jaraknya ke seluruh data training, memilih k data terdekat, lalu menetapkan kelas berdasarkan suara terbanyak di antara k tetangga tersebut. Pada praktikum ini digunakan k=5 dengan bobot uniform, sehingga setiap tetangga memberi suara yang sama, kemudian dicoba nilai k dari 3 sampai 15.

Jarak yang umum dipakai pada fitur numerik adalah jarak Euclidean, yaitu akar dari jumlah kuadrat selisih pada setiap fitur.

$$d(x, z) = \sqrt{(\Sigma_j (x_j - z_j)^2)}$$

Karena jarak dihitung dari selisih nilai mentah, fitur berskala besar dapat mendominasi, sehingga standardisasi menjadi penting; pada data ini fitur uang bernilai puluhan ribu, sedangkan skor dan helm bernilai satuan. Pemilihan k memengaruhi kompleksitas model: k yang terlalu kecil sensitif terhadap noise, sedangkan k yang terlalu besar mengaburkan pola lokal. Bansal dkk. (2022) [6] membahas KNN sebagai salah satu dari lima algoritma yang dibandingkan dalam kajian pembelajaran mesin.

Berbeda dengan Gaussian Naive Bayes yang bersifat parametrik dan mengasumsikan bentuk distribusi serta independensi fitur, KNN bersifat non-parametrik dan tidak membuat asumsi tersebut. Sebagai gantinya, KNN lebih berat saat prediksi karena harus menghitung jarak ke seluruh data training, serta peka terhadap skala dan jumlah fitur.

2.7 Evaluasi Model Klasifikasi

Evaluasi dilakukan pada data testing yang tidak dipakai saat pelatihan. Confusion matrix merangkum prediksi menjadi True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Praktikum ini menetapkan CT sebagai kelas positif (1) dan T sebagai kelas negatif (0). Accuracy adalah proporsi seluruh prediksi yang benar. Precision adalah proporsi prediksi positif yang benar, recall adalah proporsi data positif aktual yang berhasil ditemukan, dan F1-Score adalah rata-rata harmonik precision dan recall.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN) \quad F1\text{-}Score = 2 \times Precision \times Recall / (Precision + Recall)$$

Accuracy saja tidak cukup untuk menilai model sehingga perlu dibaca bersama metrik lain. Perbandingan antaralgoritma pada data yang sama dilakukan dengan metrik yang sama. Penelitian yang memakai Gaussian Naive Bayes, seperti Dani dan Ginting (2024) [1], melaporkan keempat metrik tersebut sekaligus untuk memberi gambaran kinerja yang lebih utuh.

2.8 Ketidakseimbangan Kelas dan Pemilihan Fitur

Ketidakseimbangan kelas terjadi ketika jumlah data antarkelas berbeda jauh. Kondisi ini dapat membuat model tampak akurat hanya karena condong memilih kelas mayoritas. Watono dkk. (2024)

[5] mengangkat masalah ini pada klasifikasi penerimaan deposito berjangka dan mengujinya dengan undersampling Instance Hardness Threshold serta stacking ensemble. Pada dataset ini kelas T sebanyak 58,80% dan CT sebanyak 41,20%, ketidakseimbangan yang tergolong ringan. Meski demikian, akurasi dibandingkan dengan baseline yang selalu menebak kelas mayoritas (58,80%) agar kemampuan model dapat dinilai dengan wajar, dan precision, recall, serta F1-Score dibaca untuk kedua kelas.

Pemilihan fitur dilakukan dengan menggunakan seluruh 13 kolom selain target karena semuanya merupakan informasi kondisi ronde yang tersedia pada saat pencuplikan. Tidak ada kolom yang membocorkan hasil ronde secara langsung. Kolom-kolom tersebut saling terkait (misalnya armor dan helm, atau health kedua tim yang sama-sama menurun seiring berjalannya waktu), sehingga hal ini menjadi catatan penting bagi asumsi independensi pada Naive Bayes, sedangkan bagi KNN seluruh fitur ikut menentukan jarak.

BAB III
HASIL

3.1. Lingkungan dan Dataset

Praktikum dijalankan dengan Python pada Jupyter Notebook/Google Colab menggunakan pustaka Pandas, NumPy, Matplotlib, Seaborn, dan Scikit-learn. Dataset 03_counter_strike_2.csv dibaca dengan pemisah koma dan berukuran 2.000 baris dan 14 kolom. Variabel target adalah round_winner dengan kelas T dan CT. Atribut dataset ditunjukkan pada tabel berikut.

Variabel	Keterangan
time_left	Sisa waktu ronde saat cuplikan diambil (detik).
ct_score, t_score	Skor sementara tim CT dan tim T.
map	Nama peta yang dimainkan (7 peta).
bomb_planted	Status bom: True jika sudah dipasang.
ct_health, t_health	Total health tim CT dan tim T (maksimum 500).
ct_armor, t_armor	Total armor tim CT dan tim T (maksimum 500).
ct_money, t_money	Total uang tim CT dan tim T.
ct_helmets, t_helmets	Jumlah helm yang dimiliki tim CT dan tim T.
round_winner	Target: pemenang ronde (T atau CT).

Keterangan nilai maksimum 500 pada health dan armor menunjukkan bahwa nilai tersebut merupakan akumulasi lima pemain dalam satu tim. Keterangan ini merupakan penafsiran dari rentang nilai pada data, bukan dokumentasi resmi dataset.

3.2. Penjelasan Setiap Cell Kode

- **Cell 1 – Import Library**

  Cell ini memuat seluruh pustaka. Pandas dan NumPy dipakai untuk mengolah data; Matplotlib dan Seaborn untuk grafik; Scikit-learn untuk pembagian data, preprocessing, model GaussianNB, dan metrik evaluasi. RANDOM_STATE=42 menjamin pembagian data dapat diulang dengan hasil yang sama. Dua algoritma yang digunakan adalah GaussianNB dan KNeighborsClassifier.

Import Library

```
[ ]
# 202431108
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)

import warnings
warnings.filterwarnings('ignore')

RANDOM_STATE = 42
```

- **Cell 2 – Load Dataset**
  pd.read_csv() membaca file CSV dengan pemisah koma (bawaan) dan menyimpannya pada variabel data. Cell menampilkan ukuran dataset, yaitu 2.000 baris dan 14 kolom.

Load Dataset

```
# 202431108
data = pd.read_csv('03_counter_strike_2.csv')
print("Ukuran dataset:", data.shape)
```

```
Ukuran dataset: (2000, 14)
```

- **Cell 3 – Menampilkan Lima Baris Pertama**
  data.head() menampilkan lima baris pertama untuk melihat contoh isi dan bentuk nilai tiap kolom. Kelima baris berasal dari peta de_dust2 dan semuanya dimenangkan CT. Skor CT pada baris kelima naik menjadi 1 dengan time_left kembali ke sekitar 175, yang menandakan dimulainya ronde berikutnya.

Menampilkan Lima Baris Pertama

```
# 202431108
display(data.head())
```

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	round_winner
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT

- **Cell 4 – Informasi Struktur Dataset**
  data.info() menampilkan jumlah baris, nama kolom, jumlah nilai non-null, dan tipe data. Semua kolom berisi 2.000 nilai non-null. Terdapat 11 kolom bertipe float64, 1 kolom boolean (bomb_planted), dan 2 kolom teks (map dan round_winner). Pada versi Pandas yang lebih lama, kolom teks ditampilkan sebagai object.

Informasi Struktur Dataset

```
# 202431108
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   time_left     2000 non-null   float64
 1   ct_score      2000 non-null   float64
 2   t_score       2000 non-null   float64
 3   map           2000 non-null   object
 4   bomb_planted  2000 non-null   bool
 5   ct_health     2000 non-null   float64
 6   t_health      2000 non-null   float64
 7   ct_armor      2000 non-null   float64
 8   t_armor       2000 non-null   float64
 9   ct_money      2000 non-null   float64
 10  t_money       2000 non-null   float64
 11  ct_helmets    2000 non-null   float64
 12  t_helmets     2000 non-null   float64
 13  round_winner  2000 non-null   object
dtypes: bool(1), float64(11), object(2)
memory usage: 205.2+ KB
```

- **Cell 5 – Statistik Deskriptif**
  data.describe() meringkas kolom numerik. Rata-rata time_left adalah 95,63 detik dengan rentang 0,05 sampai 175. Rata-rata skor CT 6,84 dan skor T 6,29, dengan nilai maksimum 15. Health tiap tim rata-rata sekitar 406 (CT) dan 403 (T), dan armor sekitar 307 (CT) dan 310 (T). Perbedaan skala paling mencolok terdapat pada uang: ct_money bernilai rata-rata 7.867,35 (maksimum 58.050) dan t_money 12.377,60 (maksimum 67.700), sedangkan fitur lain berskala satuan hingga ratusan. Perbedaan skala inilah yang mendasari penggunaan standardisasi.

Statistik Deskriptif

```
[5]
✓ 0d
# 202431108
print(data.describe().T.round(2).to_string())
```

```
              count      mean       std   min      25%      50%      75%      max
time_left    2000.0     95.63     53.73  0.05    54.91    94.91   156.43    175.0
ct_score     2000.0      6.84      4.23  0.00     3.00     7.00    10.00     15.0
t_score      2000.0      6.29      4.19  0.00     3.00     6.00    10.00     15.0
ct_health    2000.0    406.17    137.62  2.00   315.75   500.00   500.00    500.0
t_health     2000.0    402.99    137.70  0.00   328.75   500.00   500.00    500.0
ct_armor     2000.0    306.75    172.86  0.00   188.00   300.00   488.00    500.0
t_armor      2000.0    309.54    168.69  0.00   190.00   362.00   467.00    500.0
ct_money     2000.0   7867.35   9017.49  0.00  1200.00  4000.00  11762.50  58050.0
t_money      2000.0  12377.60  12209.58  0.00  1700.00  9075.00  19250.00  67700.0
ct_helmets   2000.0      1.82      1.77  0.00     0.00     1.00     3.00      5.0
t_helmets    2000.0      3.01      1.96  0.00     1.00     4.00     5.00      5.0
```

- **Cell 6 – Pemeriksaan Missing Value dan Duplikasi**
  Cell menghitung nilai kosong dengan isnull().sum() dan baris duplikat dengan duplicated().sum(). Hasilnya tidak ada missing value pada seluruh kolom, tetapi terdapat 39 baris duplikat (1,95%). Baris ini tidak dihapus karena nilai fitur yang sama pada dua cuplikan merupakan kemungkinan yang wajar pada data keadaan ronde, dan jumlahnya kecil. Konsekuensinya, sebagian baris identik dapat berada pada data training dan testing sekaligus, sehingga akurasi bisa sedikit lebih optimistis.

Pemeriksaan Missing Value dan Duplikasi

```
[6]
✓ 0d
# 202431108
print("Jumlah Missing Value:")
print(data.isnull().sum())

print("\nJumlah Data Duplikat:")
print(data.duplicated().sum())
```

```
Jumlah Missing Value:
time_left       0
ct_score        0
t_score         0
map             0
bomb_planted    0
ct_health       0
t_health        0
ct_armor        0
t_armor         0
ct_money        0
t_money         0
ct_helmets      0
t_helmets       0
round_winner    0
dtype: int64

Jumlah Data Duplikat:
39
```

- **Cell 7 – Identifikasi Fitur Numerik dan Kategorikal**
  select_dtypes(include=np.number) memilih kolom numerik dan select_dtypes(exclude=np.number) memilih kolom non-numerik. Hasilnya adalah 11 fitur numerik dan 3 kolom non-numerik, yaitu map, bomb_planted (boolean), dan round_winner (target). Kolom bomb_planted dan map akan ditangani secara khusus pada tahap berikutnya.



- **Cell 8 – Melihat Nilai Unik Fitur Non-Numerik**
  Perulangan memakai unique() untuk melihat kategori pada kolom non-numerik. Kolom map memiliki 7 peta (de_dust2, de_mirage, de_nuke, de_inferno, de_overpass, de_vertigo, de_train), bomb_planted bernilai False atau True, dan round_winner bernilai CT atau T. Seluruh kategori tampak bersih tanpa penulisan ganda.



- **Cell 9 – EDA Distribusi Target**

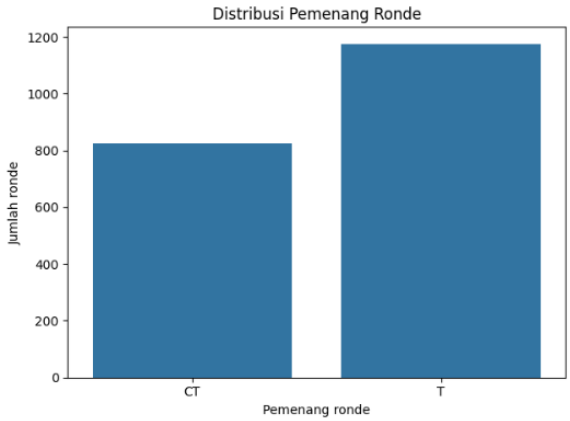
  value_counts() menghitung jumlah kelas, sedangkan normalize=True menghasilkan persentasenya. Kelas T berjumlah 1.176 (58,80%) dan CT berjumlah 824 (41,20%). Countplot menunjukkan T lebih banyak, tetapi selisihnya tidak ekstrem sehingga ketidakseimbangan tergolong ringan. Baseline yang selalu menebak T akan mencapai akurasi sekitar 58,8%, dan angka ini dijadikan pembanding bagi model.

EDA Distribusi Target

```
# 202431108
print(data['round_winner'].value_counts())
print((data['round_winner'].value_counts(normalize=True) * 100).round(2))

sns.countplot(data=data, x='round_winner')
plt.title('Distribusi Pemenang Ronde')
plt.xlabel('Pemenang ronde')
plt.ylabel('Jumlah ronde')
plt.tight_layout()
plt.show()
```

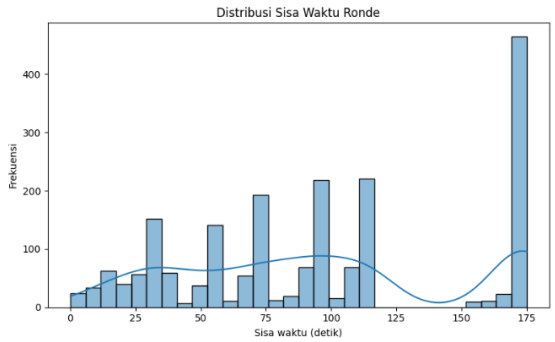
```
round_winner
T     1176
CT     824
Name: count, dtype: int64
round_winner
T     58.8
CT    41.2
Name: proportion, dtype: float64
```



- **Cell 10 – EDA Distribusi Sisa Waktu Ronde**
  Histogram dengan 30 interval dan kurva KDE menampilkan sebaran sisa waktu ronde. Sebaran tidak berbentuk lonceng. Terdapat lonjakan di dekat 175 detik, yaitu 380 baris dengan time_left di atas 170, yang sesuai dengan cuplikan di awal ronde. Selebihnya nilai tersebar pada kelipatan sekitar 20 detik hingga mendekati nol. Median time_left adalah 94,91 detik. Bentuk sebaran yang jauh dari Gaussian ini perlu dicatat karena Gaussian Naive Bayes mengasumsikan distribusi normal.

EDA Distribusi Sisa Waktu Ronde

```
# 202431108
plt.figure(figsize=(8, 5))
sns.histplot(data=data, x='time_left', bins=30, kde=True)
plt.title('Distribusi Sisa Waktu Ronde')
plt.xlabel('Sisa waktu (detik)')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
print(data['time_left'].describe().round(2))
```




```
count    2000.00
mean       95.63
std        53.73
min         0.05
25%        54.91
50%        94.91
75%       156.43
max       175.00
Name: time_left, dtype: float64
```

- **Cell 11 – EDA Pemenang Ronde pada Setiap Peta dan Status Bom**
  Cell menampilkan tabel silang pemenang ronde menurut peta dan menurut status bom, disertai grafik batang. Tim T unggul pada enam peta; satu-satunya pengecualian adalah de_train, dengan CT 78 kemenangan dan T 48. Pada peta de_mirage tercatat 234 kemenangan CT dan 319 kemenangan T. Status bom tampak berpengaruh kuat: ketika bom belum dipasang CT menang sebesar 44,5%, sedangkan ketika bom sudah dipasang kemenangan CT hanya 19,3% dan T 80,7%. Sebanyak 264 baris (13,2%) bomnya sudah dipasang. Hal ini menunjukkan bahwa bomb_planted dan map memuat informasi yang berguna bagi model.

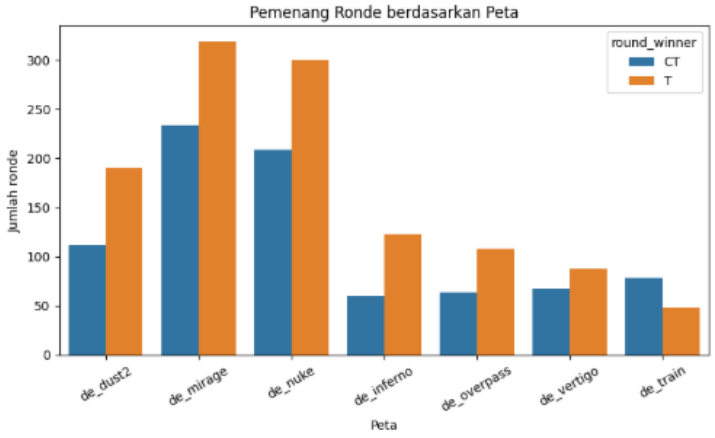
EDA Pemenang Ronde pada Setiap Peta dan Status Bom

```
# 202431108
print(pd.crosstab(data['map'], data['round_winner']))
print()
print(pd.crosstab(data['bomb_planted'], data['round_winner'],
                  normalize='index').round(3))

plt.figure(figsize=(8, 5))
sns.countplot(data=data, x='map', hue='round_winner')
plt.title('Pemenang Ronde berdasarkan Peta')
plt.xlabel('Peta')
plt.ylabel('Jumlah ronde')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

```
round_winner   CT    T
map
de_dust2      112  190
de_inferno     60  123
de_mirage     234  319
de_nuke       209  300
de_overpass    64  108
de_train       78   48
de_vertigo     67   88

round_winner     CT      T
bomb_planted
False         0.445  0.555
True          0.193  0.807
```

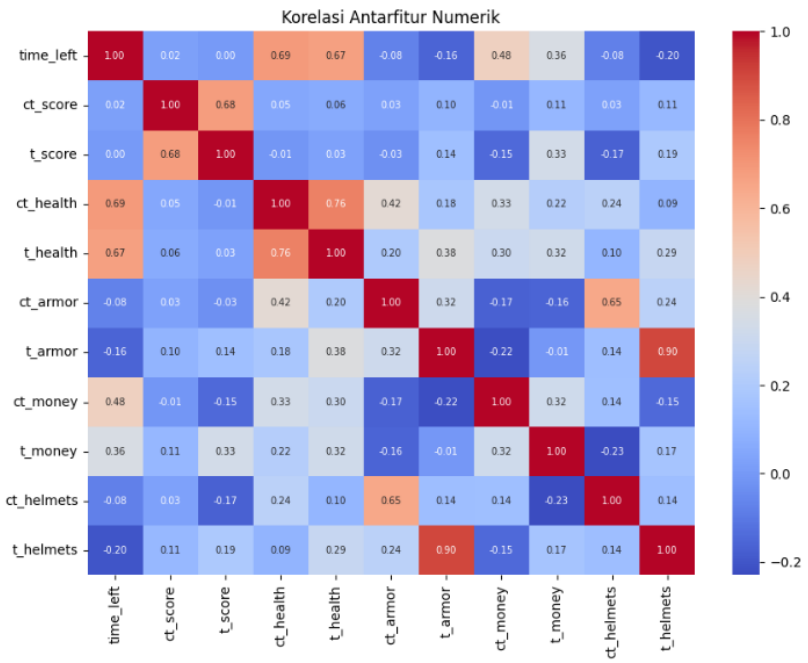


- **Cell 12 – EDA Korelasi Antarfitur Numerik**

  Heatmap menampilkan korelasi Pearson antarfitur numerik. Beberapa pasangan berkorelasi tinggi: t_armor dengan t_helmets (0,90), ct_health dengan t_health (0,76), ct_score dengan t_score (0,68), time_left dengan ct_health (0,69) dan t_health (0,67), serta ct_armor dengan ct_helmets (0,65). Karena Naive Bayes memperlakukan fitur sebagai saling bebas bila kelas diketahui, korelasi setinggi ini menunjukkan bahwa asumsi tersebut tidak terpenuhi sepenuhnya, dan hal ini kemungkinan membatasi kinerja model. Korelasi ini dihitung dari seluruh data, sedangkan asumsi Naive Bayes berlaku bersyarat pada kelas, sehingga angkanya hanya indikasi awal.

EDA Korelasi Antarfitur Numerik

```
# 202431108
plt.figure(figsize=(9, 7))
sns.heatmap(data[numerical_features].corr(), annot=True, fmt='.2f',
            cmap='coolwarm', annot_kws={'size': 7})
plt.title('Korelasi Antarfitur Numerik')
plt.tight_layout()
plt.show()
```



- **Cell 13 – Menentukan Fitur dan Target**

  X dibuat dengan menghapus kolom round_winner. Kolom bomb_planted diubah dari boolean ke 0/1, sedangkan target y dikodekan T=0 dan CT=1 sehingga CT menjadi kelas positif. Hasilnya 13 fitur prediktor dan distribusi target 1.176 untuk kelas 0 dan 824 untuk kelas 1. Seluruh fitur merupakan kondisi ronde sehingga tidak ada kolom yang membocorkan target.

Menentukan Fitur dan Target

```
# 202431108
X = data.drop(columns=['round_winner'])
X['bomb_planted'] = X['bomb_planted'].astype(int)
y = data['round_winner'].map({'T': 0, 'CT': 1})

print("Fitur prediktor:", X.columns.tolist())
print("\nDistribusi target:")
print(y.value_counts())
```

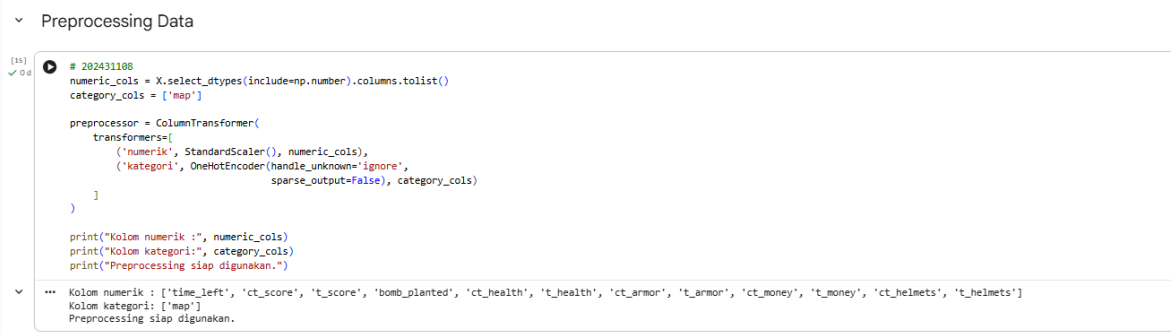
```
Fitur prediktor: ['time_left', 'ct_score', 't_score', 'map', 'bomb_planted', 'ct_health', 't_health', 'ct_armor', 't_armor', 'ct_money', 't_money', 'ct_helmets', 't_helmets']

Distribusi target:
round_winner
0    1176
1     824
Name: count, dtype: int64
```

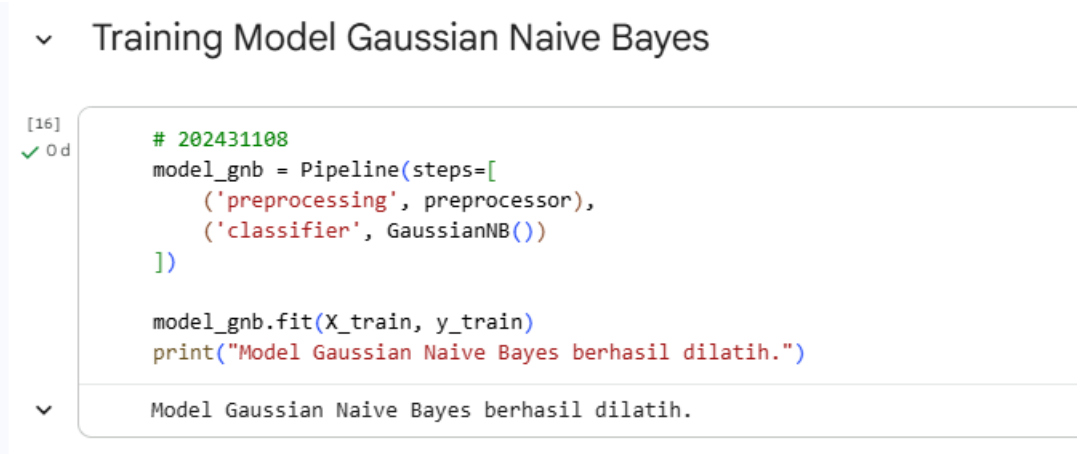
- **Cell 14 – Membagi Data Training dan Testing**
  train_test_split membagi data menjadi 80% training dan 20% testing. random_state=42 membuat pembagian dapat diulang dan stratify=y menjaga proporsi kelas. Hasilnya 1.600 data training dan 400 data testing, masing-masing dengan 13 fitur. Pembagian yang sama dipakai untuk KNN agar perbandingan kedua algoritma adil.



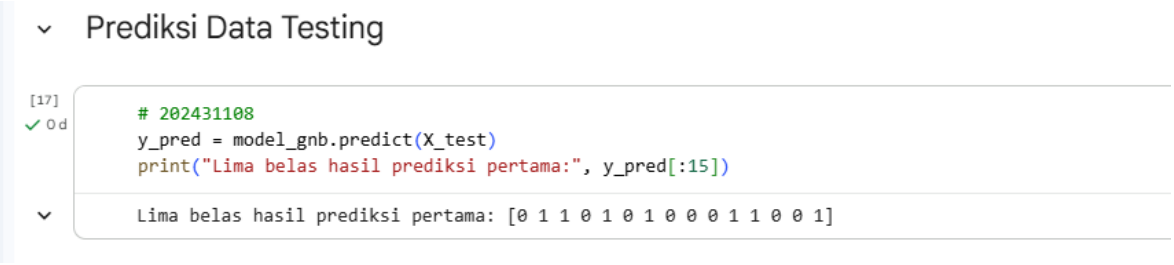
- **Cell 15 – Preprocessing Data**
  ColumnTransformer menerapkan transformasi sesuai tipe kolom. Kolom numerik (termasuk bomb_planted yang sudah berupa 0/1) distandardisasi dengan StandardScaler, sedangkan kolom map diubah dengan OneHotEncoder. handle_unknown='ignore' mencegah galat apabila ada peta yang tidak ditemukan saat training. Setelah transformasi terdapat 19 fitur, yaitu 12 fitur numerik dan 7 kolom indikator peta. Perlu dicatat bahwa kolom indikator bersifat biner dan tidak berdistribusi Gaussian, sehingga pemakaiannya pada GaussianNB merupakan pendekatan praktis yang kurang ideal secara teoretis.



- **Cell 16 – Training Model Gaussian Naive Bayes**
  Pipeline menggabungkan preprocessing dengan GaussianNB. Saat fit(X_train, y_train) dijalankan, transformasi dipelajari dari data training, lalu GaussianNB menghitung probabilitas prior setiap kelas serta rata-rata dan varians setiap fitur pada masing-masing kelas. Model tidak melakukan iterasi optimasi, sehingga pelatihan berlangsung sangat cepat.



- **Cell 17 – Prediksi Data Testing**
  model_gnb.predict(X_test) menghitung posterior setiap kelas untuk tiap baris data testing dan memilih kelas dengan nilai terbesar. Label 0 berarti T menang dan 1 berarti CT menang. Cell menampilkan 15 prediksi pertama, yaitu [0 1 1 0 1 0 1 0 0 0 1 1 0 0 1].



- **Cell 18 – Evaluasi Model**
  Cell menghitung Accuracy, Precision, Recall, dan F1-Score dengan membandingkan y_test dan y_pred, lalu menampilkan classification report. Hasilnya Accuracy 72,00%; untuk kelas CT (positif) Precision 65,32%, Recall 68,48%, dan F1-Score 66,86%. Untuk kelas T diperoleh precision 0,77, recall 0,74, dan F1-score 0,76. Akurasi 72,00% berada sekitar 13 poin persentase di atas baseline mayoritas (58,8%), sehingga model memang menangkap pola dari fitur. Kinerja kelas T sedikit lebih baik daripada CT, yang wajar karena T memiliki lebih banyak data latih.

Evaluasi Model

[18] ✓ 0d

```
# 202431108
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall   :", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score :", f1_score(y_test, y_pred, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    labels=[0, 1], target_names=['T', 'CT'],
    zero_division=0
))
```

```
Accuracy : 0.72
Precision: 0.653179190751445
Recall   : 0.6848484848484848
F1-Score : 0.6686390532544378

Classification Report:
              precision    recall  f1-score   support

           T       0.77      0.74      0.76       235
          CT       0.65      0.68      0.67       165

    accuracy                           0.72       400
   macro avg       0.71      0.71      0.71       400
weighted avg       0.72      0.72      0.72       400
```

- **Cell 19 – Confusion Matrix**

  Confusion matrix membandingkan label aktual dengan prediksi. Hasilnya TN=175, FP=60, FN=52, dan TP=113. TN adalah ronde T yang diprediksi T, FP adalah ronde T yang diprediksi CT, FN adalah ronde CT yang diprediksi T, dan TP adalah ronde CT yang diprediksi CT. Sebanyak 175 dari 235 ronde T (74,47%) dan 113 dari 165 ronde CT (68,48%) diprediksi benar. Total kesalahan adalah 112 dari 400 data (28%). Jumlah FP dan FN relatif seimbang, sehingga model tidak condong ekstrem ke salah satu kelas.

Confusion Matrix

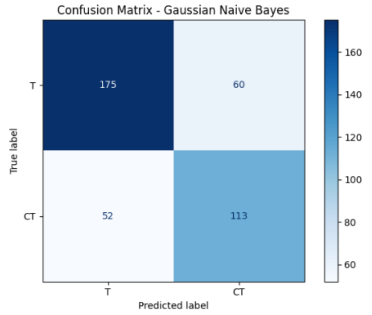
[19] ✓ 0d

```
# 202431108
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
print("Confusion Matrix Gaussian Naive Bayes:")
print(cm)

tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['T', 'CT']
).plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix - Gaussian Naive Bayes')
plt.tight_layout()
plt.show()
```

```
Confusion Matrix Gaussian Naive Bayes:
[[175  60]
 [ 52 113]]
TN=175, FP=60, FN=52, TP=113
```



- **Cell 20 – Parameter Hasil Pelatihan Gaussian Naive Bayes**
  Cell menampilkan parameter yang dipelajari model. Prior kelas adalah 0,5881 untuk T dan 0,4119 untuk CT, sesuai dengan 941 dan 659 data training. Pada nilai rata-rata fitur terstandardisasi, ronde yang dimenangkan CT memiliki ct_health (0,218 berbanding −0,153) dan ct_armor (0,347 berbanding −0,243) yang lebih tinggi, sedangkan ronde yang dimenangkan T lebih sering memiliki bom terpasang (0,150 berbanding −0,214). Pola ini sejalan dengan hasil EDA. Akurasi data training adalah 72,06%, hampir sama dengan akurasi testing 72,00%. Kesenjangan yang nyaris tidak ada ini menunjukkan model tidak overfitting, dan batas kinerjanya kemungkinan lebih berkaitan dengan kesederhanaan asumsi model, termasuk independensi fitur, daripada dengan kekurangan data.

Parameter Hasil Pelatihan Gaussian Naive Bayes

[20] ✓ 0d

```
# 202431108
gnb = model_gnb.named_steps['classifier']
feature_names = model_gnb.named_steps['preprocessing'].get_feature_names_out()

print("Jumlah fitur setelah preprocessing:", len(feature_names))
print("Prior kelas [T, CT]:", gnb.class_prior_.round(4))
print("Jumlah data latih per kelas:", gnb.class_count_)

params = pd.DataFrame(gnb.theta_, index=['T', 'CT'], columns=feature_names)
print("\nRata-rata (theta) beberapa fitur per kelas:")
params.columns = [c.split('__')[1] for c in params.columns]
print(params[['ct_health', 't_health', 'ct_armor', 'bomb_planted']].round(3).to_string())
print("\nAccuracy data training:", accuracy_score(y_train, model_gnb.predict(X_train)))
```

```
Jumlah fitur setelah preprocessing: 19
Prior kelas [T, CT]: [0.5881 0.4119]
Jumlah data latih per kelas: [941. 659.]

Rata-rata (theta) beberapa fitur per kelas:
    ct_health  t_health  ct_armor  bomb_planted
T      -0.153     0.077    -0.243         0.150
CT      0.218    -0.111     0.347        -0.214

Accuracy data training: 0.720625
```

- **Cell 21 – Pengujian Nilai var_smoothing**
  Cell menguji lima nilai var_smoothing (10^{-9}, 10^{-6}, 10^{-3}, 10^{-1}, dan 1) dan menggambarkan pengaruhnya terhadap accuracy. Hasil accuracy berturut-turut 72,00%; 72,00%; 72,25%; 72,75%; dan 74,00%, sedangkan F1-Score kelas CT 66,86%; 66,86%; 67,26%; 68,04%; dan 66,88%. Peningkatan terjadi seiring membesarnya penghalusan, tetapi selisihnya kecil. Dengan 400 data uji, galat baku accuracy sekitar ±2,2 poin persentase, sehingga kenaikan sebesar 2 poin belum tentu merupakan perbaikan sejati. Nilai terbaik juga dipilih berdasarkan data testing yang sama, sehingga hasilnya cenderung optimistis. Oleh karena itu, hasil model dengan pengaturan bawaan (Accuracy 72,00%) dijadikan hasil utama laporan ini.

Pengujian Nilai var_smoothing

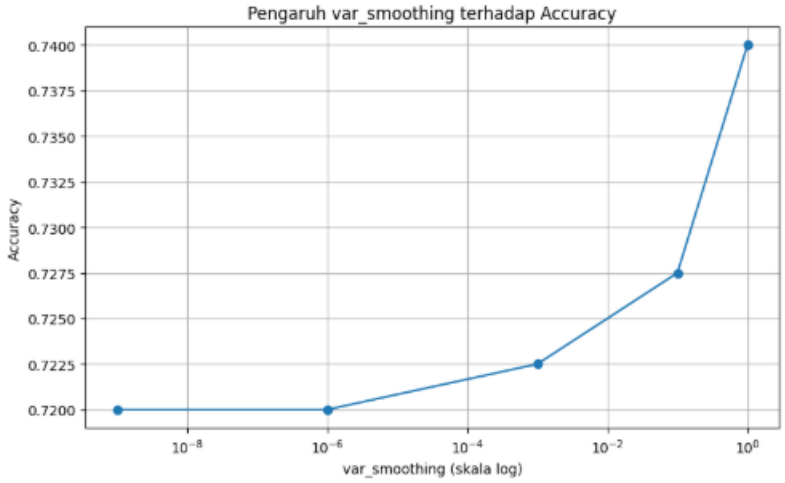
```
# 202431108
smoothing_values = [1e-9, 1e-6, 1e-3, 1e-1, 1]
hasil = []

for vs in smoothing_values:
    model_temp = Pipeline(steps=[
        ('preprocessing', preprocessor),
        ('classifier', GaussianNB(var_smoothing=vs))
    ])
    model_temp.fit(X_train, y_train)
    pred_temp = model_temp.predict(X_test)
    hasil.append([f'{vs:.0e}', accuracy_score(y_test, pred_temp),
                  f1_score(y_test, pred_temp)])

hasil_df = pd.DataFrame(hasil, columns=['var_smoothing', 'accuracy', 'f1_ct'])
print(hasil_df.round(4).to_string(index=False))

plt.figure(figsize=(8, 5))
plt.semilogx(smoothing_values, hasil_df['accuracy'], marker='o')
plt.title('Pengaruh var_smoothing terhadap Accuracy')
plt.xlabel('var_smoothing (skala log)')
plt.ylabel('Accuracy')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
var_smoothing  accuracy  f1_ct
        1e-09    0.7200 0.6686
        1e-06    0.7200 0.6686
        1e-03    0.7225 0.6726
        1e-01    0.7275 0.6804
        1e+00    0.7400 0.6688
```



- **Cell 22 – Preprocessing untuk KNN**
  Preprocessing KNN dibuat sebagai objek terpisah (preprocessor_knn) agar tidak bercampur dengan Naive Bayes, tetapi isinya sama: StandardScaler untuk fitur numerik dan

OneHotEncoder untuk kolom map. Data training dan testing memakai pembagian yang sama dengan Gaussian Naive Bayes (1.600 dan 400 baris, 13 fitur) sehingga perbandingan adil. Standardisasi sangat penting bagi KNN karena jarak dihitung dari nilai fitur; tanpa itu, ct_money dan t_money yang bernilai puluhan ribu akan mendominasi jarak.

Preprocessing untuk KNN

```
[22]
✓ 0 d
# 202431108
# Fitur dan pembagian data sama dengan Gaussian Naive Bayes agar perbandingan adil
preprocessor_knn = ColumnTransformer(
    transformers=[
        ('numerik', StandardScaler(), numeric_cols),
        ('kategori', OneHotEncoder(handle_unknown='ignore',
                                   sparse_output=False), category_cols)
    ]
)

print("X_train KNN:", X_train.shape)
print("X_test KNN :", X_test.shape)
print("Preprocessing KNN siap digunakan.")
```

```
X_train KNN: (1600, 13)
X_test KNN : (400, 13)
Preprocessing KNN siap digunakan.
```

- **Cell 23 – Training Model KNN dengan k=5**

  Pipeline menggabungkan preprocessing dengan KNeighborsClassifier (n_neighbors=5, weights='uniform'). Model mempertimbangkan lima tetangga terdekat dengan bobot suara yang sama. Pada KNN, fit() hanya menyimpan data training yang sudah ditransformasi dan tidak mempelajari parameter, sehingga pekerjaan hitung baru terjadi saat prediksi.

Training Model KNN dengan k=5

```
[23]
✓ 0 d
# 202431108
model_knn = Pipeline(steps=[
    ('preprocessing', preprocessor_knn),
    ('classifier', KNeighborsClassifier(n_neighbors=5, weights='uniform'))
])

model_knn.fit(X_train, y_train)
print("Model KNN dengan k=5 berhasil dilatih.")
```

```
Model KNN dengan k=5 berhasil dilatih.
```

- **Cell 24 – Prediksi Data Testing KNN**

  model_knn.predict(X_test) menghitung jarak setiap data testing ke 1.600 data training, mengambil lima tetangga terdekat, lalu memilih kelas mayoritas. Nilai 0 berarti T dan 1 berarti CT. Lima belas prediksi pertama adalah [1 1 0 0 1 0 1 0 0 1 0 0 0 1 1].

Prediksi Data Testing KNN

```
[24]
✓ 0 d
# 202431108
y_pred_knn = model_knn.predict(X_test)
print("Lima belas hasil prediksi pertama:", y_pred_knn[:15])
```

```
Lima belas hasil prediksi pertama: [1 1 0 0 1 0 1 0 0 1 0 0 0 1 1]
```

- **Cell 25 – Evaluasi Model KNN**
  Cell menghitung metrik evaluasi KNN dengan k=5. Hasilnya Accuracy 86,25%; untuk kelas CT Precision 86,67%, Recall 78,79%, dan F1-Score 82,54%. Untuk kelas T diperoleh precision 0,86, recall 0,91, dan F1-score 0,89. Seluruh metrik ini lebih tinggi daripada Gaussian Naive Bayes (Accuracy 72,00%, Recall CT 68,48%, F1-Score CT 66,86%).

Evaluasi Model KNN

[25] 0 d

```
# 202431108
print("Accuracy :", accuracy_score(y_test, y_pred_knn))
print("Precision:", precision_score(y_test, y_pred_knn, zero_division=0))
print("Recall   :", recall_score(y_test, y_pred_knn, zero_division=0))
print("F1-Score :", f1_score(y_test, y_pred_knn, zero_division=0))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_knn,
    labels=[0, 1], target_names=['T', 'CT'],
    zero_division=0
))
```

```
Accuracy : 0.86
Precision: 0.8609271523178808
Recall   : 0.7878787878787878
F1-Score : 0.8227848101265823

Classification Report:
              precision    recall  f1-score   support

           T       0.86      0.91      0.88       235
          CT       0.86      0.79      0.82       165

    accuracy                           0.86       400
   macro avg       0.86      0.85      0.85       400
weighted avg       0.86      0.86      0.86       400
```

- **Cell 26 – Confusion Matrix KNN**
  Confusion matrix KNN menghasilkan TN=215, FP=20, FN=35, dan TP=130. Sebanyak 215 dari 235 ronde T (91,49%) dan 130 dari 165 ronde CT (78,79%) diprediksi benar, dengan total kesalahan 55 dari 400 data (13,75%). Dibandingkan Naive Bayes (FP=60, FN=52), KNN menghasilkan jauh lebih sedikit prediksi positif palsu dan sedikit lebih sedikit negatif palsu. Kesalahan KNN lebih banyak berupa ronde CT yang diprediksi T daripada sebaliknya.

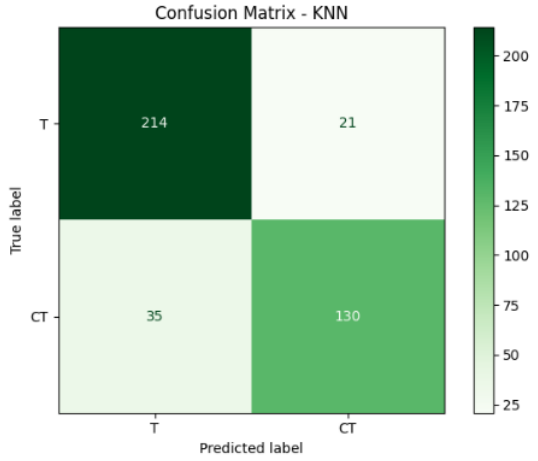
Confusion Matrix KNN

```
[26]
✓ 0d
# 202431108
cm_knn = confusion_matrix(y_test, y_pred_knn, labels=[0, 1])
print("Confusion Matrix KNN:")
print(cm_knn)

tn, fp, fn, tp = cm_knn.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

ConfusionMatrixDisplay(
    confusion_matrix=cm_knn,
    display_labels=['T', 'CT']
).plot(cmap='Greens', values_format='d')
plt.title('Confusion Matrix - KNN')
plt.tight_layout()
plt.show()
```

```
Confusion Matrix KNN:
[[214  21]
 [ 35 130]]
TN=214, FP=21, FN=35, TP=130
```



- **Cell 27 – Menguji Beberapa Nilai k pada KNN**
  Cell mentransformasikan data training dan testing satu kali dengan preprocessor_knn (scaler dan encoder di-fit hanya pada data training), lalu menguji k=3, 5, 7, 9, 11, 13, dan 15. Accuracy berturut-turut 88,00%; 86,25%; 83,25%; 81,00%; 79,50%; 80,75%; dan 78,75%. Accuracy menurun seiring bertambahnya k. Pola ini menunjukkan bahwa informasi lokal dari data yang sangat mirip penting pada dataset ini, sedangkan k yang besar mengaburkannya. Nilai terbaik berada di ujung bawah rentang yang diuji, sehingga k yang lebih kecil tidak diketahui hasilnya.

Menguji Beberapa Nilai k pada KNN

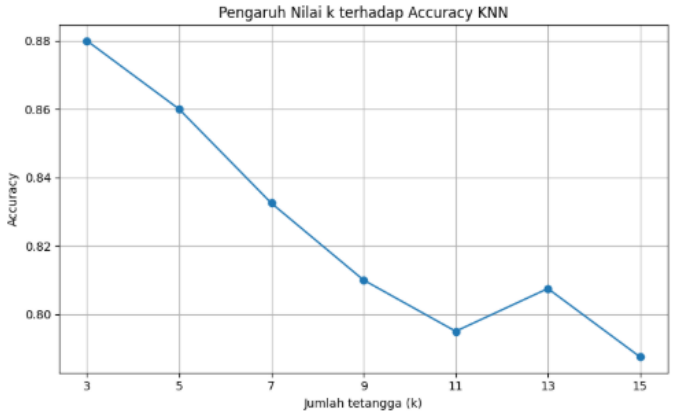
```
# 202431108
# Data ditransformasikan satu kali; scaler/encoder hanya di-fit pada training
X_train_t = preprocessor_knn.fit_transform(X_train)
X_test_t = preprocessor_knn.transform(X_test)

k_values = range(3, 16, 2)
accuracies = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_t, y_train)
    pred_temp = knn_temp.predict(X_test_t)
    accuracies.append(accuracy_score(y_test, pred_temp))

plt.figure(figsize=(8, 5))
plt.plot(list(k_values), accuracies, marker='o')
plt.title('Pengaruh Nilai k terhadap Accuracy KNN')
plt.xlabel('Jumlah tetangga (k)')
plt.ylabel('Accuracy')
plt.xticks(list(k_values))
plt.grid(True)
plt.tight_layout()
plt.show()

print("Accuracy setiap k:", {k: round(float(a), 4) for k, a in zip(k_values, accuracies)})
```




```
Accuracy setiap k: {3: 0.88, 5: 0.86, 7: 0.8325, 9: 0.81, 11: 0.795, 13: 0.8075, 15: 0.7875}
```

- **Cell 28 – Evaluasi KNN dengan Nilai k Terbaik**
  Cell memilih k dengan accuracy tertinggi memakai argmax, yaitu k=3, lalu melatih ulang model dan menampilkan classification report. Hasilnya Accuracy 88,00%; untuk kelas CT precision 0,87, recall 0,84, dan F1-score 0,85, sedangkan kelas T memperoleh 0,89; 0,91; dan 0,90. Karena k dipilih berdasarkan data testing yang sama, hasil ini cenderung optimistis. Oleh karena itu, hasil dengan k=5 (pengaturan awal) dipakai sebagai hasil utama KNN, dan k=3 sebagai hasil pengujian lanjutan.

Evaluasi KNN dengan Nilai k Terbaik

```
# 202431108
best_k = list(k_values)[int(np.argmax(accuracies))]
print("Nilai k dengan accuracy tertinggi pada percobaan:", best_k)

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train_t, y_train)
best_pred = best_knn.predict(X_test_t)

print(classification_report(
    y_test, best_pred,
    labels=[0, 1], target_names=['T', 'CT'],
    zero_division=0
))
```

```
Nilai k dengan accuracy tertinggi pada percobaan: 3
              precision    recall  f1-score   support

           T       0.89      0.91      0.90       235
          CT       0.87      0.84      0.85       165

    accuracy                           0.88       400
   macro avg       0.88      0.87      0.88       400
weighted avg       0.88      0.88      0.88       400
```

- **Cell 29 – Perbandingan Gaussian Naive Bayes dan KNN**

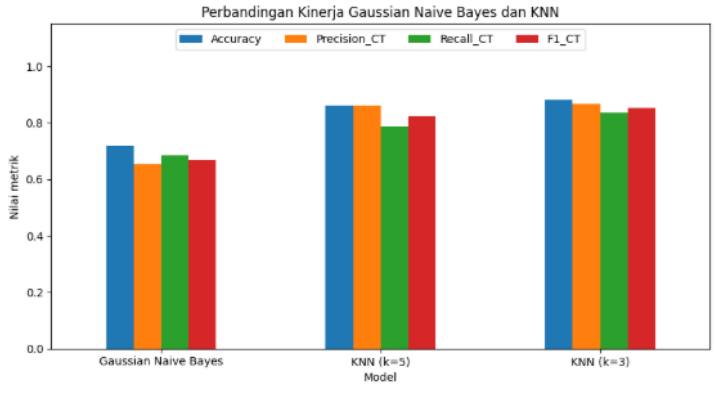
  Cell menyusun tabel dan grafik perbandingan. Gaussian Naive Bayes: Accuracy 72,00%, Precision CT 65,32%, Recall CT 68,48%, F1 66,86%. KNN k=5: 86,25%; 86,67%; 78,79%; 82,54%. KNN k=3: 88,00%; 86,79%; 83,64%; 85,19%. KNN unggul sekitar 14 poin persentase accuracy, selisih yang jauh melebihi galat baku sekitar ±2,2 poin pada 400 data uji. Salah satu penjelasan yang masuk akal adalah bahwa Naive Bayes dibatasi oleh asumsi independensi dan distribusi Gaussian yang tidak terpenuhi (Cell 10 dan 12), sedangkan KNN tidak membuat asumsi tersebut. Namun ada catatan penting: data berupa cuplikan dari ronde yang sama, dan pembagian acak memungkinkan cuplikan yang sangat mirip berada di data training sekaligus testing. Kombinasi map, ct_score, dan t_score kemungkinan hampir mengidentifikasi satu ronde. KNN yang mengandalkan tetangga terdekat dapat diuntungkan oleh kemiripan ini, sehingga akurasinya mungkin lebih tinggi daripada pada ronde yang benar-benar baru. Karena dataset tidak memiliki kolom identitas ronde, hal ini tidak dapat diverifikasi dan perlu diuji lebih lanjut dengan pembagian per ronde.

Perbandingan Gaussian Naive Bayes dan KNN

```
# 202431108
perbandingan = pd.DataFrame({
    'Model': ['Gaussian Naive Bayes', 'KNN (k=5)', f'KNN (k={best_k})'],
    'Accuracy': [accuracy_score(y_test, y_pred),
                 accuracy_score(y_test, y_pred_knn),
                 accuracy_score(y_test, best_pred)],
    'Precision_CT': [precision_score(y_test, y_pred),
                     precision_score(y_test, y_pred_knn),
                     precision_score(y_test, best_pred)],
    'Recall_CT': [recall_score(y_test, y_pred),
                  recall_score(y_test, y_pred_knn),
                  recall_score(y_test, best_pred)],
    'F1_CT': [f1_score(y_test, y_pred),
              f1_score(y_test, y_pred_knn),
              f1_score(y_test, best_pred)]
})
print(perbandingan.round(4).to_string(index=False))

perbandingan.set_index('Model').plot(kind='bar', figsize=(9, 5))
plt.title('Perbandingan Kinerja Gaussian Naive Bayes dan KNN')
plt.ylabel('Nilai metrik')
plt.ylim(0, 1.15)
plt.xticks(rotation=0)
plt.legend(loc='upper center', ncol=4)
plt.tight_layout()
plt.show()
```

```
                Model  Accuracy  Precision_CT  Recall_CT  F1_CT
Gaussian Naive Bayes      0.72        0.6532     0.6848 0.6686
          KNN (k=5)       0.86        0.8609     0.7879 0.8228
          KNN (k=3)       0.88        0.8679     0.8364 0.8519
```



BAB IV
PENUTUP

- Berdasarkan pengolahan dataset Counter-Strike, proses klasifikasi memerlukan EDA, preprocessing, pembagian data, pelatihan model, dan evaluasi. Dataset terdiri dari 2.000 baris dan 14 kolom tanpa missing value, dengan 39 baris duplikat (1,95%) yang dipertahankan. Target round_winner tidak sepenuhnya seimbang: kelas T sebesar 58,80% dan CT sebesar 41,20%.

- Dua algoritma diuji dengan pembagian data dan preprocessing yang sama. Gaussian Naive Bayes memperoleh accuracy 72,00% dengan precision 65,32%, recall 68,48%, dan F1-Score 66,86% untuk kelas CT sebagai kelas positif (TN=175, FP=60, FN=52, TP=113). KNN dengan k=5 memperoleh accuracy 86,25% dengan precision 86,67%, recall 78,79%, dan F1-Score 82,54% (TN=215, FP=20, FN=35, TP=130). Pengujian beberapa nilai k menunjukkan accuracy tertinggi 88,00% pada k=3, tetapi nilai ini dipilih dari data testing sehingga bersifat optimistis. Pada eksperimen ini KNN lebih unggul pada seluruh metrik, dengan selisih accuracy sekitar 14 poin persentase.

- Keterbatasan Gaussian Naive Bayes berkaitan dengan asumsinya. Beberapa fitur berkorelasi tinggi (misalnya t_armor dengan t_helmets sebesar 0,90), distribusi time_left jauh dari normal, dan kolom indikator peta bersifat biner. Pengujian var_smoothing hanya menaikkan accuracy hingga 74,00%, selisih yang masih dalam kisaran galat baku pada 400 data uji. Sebaliknya KNN tidak membuat asumsi tersebut. Namun keunggulan KNN perlu dibaca hati-hati karena data berupa cuplikan dari ronde yang sama dan pembagian acak memungkinkan cuplikan yang sangat mirip berada pada data training dan testing, sehingga akurasinya mungkin lebih tinggi daripada pada ronde yang benar-benar baru. Pengujian lanjutan dengan pembagian berdasarkan ronde, validasi silang, dan penanganan fitur berkorelasi diperlukan sebelum hasil ini dijadikan dasar keputusan.

DAFTAR PUSTAKA

[1] Y. Dani and M. A. Ginting, “Comparison of Iris dataset classification with Gaussian naïve Bayes and decision tree algorithms,” International Journal of Electrical and Computer Engineering (IJECE), vol. 14, no. 2, pp. 1959–1968, 2024. https://doi.org/10.11591/ijece.v14i2.pp1959-1968

[2] P. Srisuradetchai, “Posterior averaging with Gaussian naive Bayes and the R package RandomGaussianNB for big-data classification,” Frontiers in Big Data, vol. 8, Art. no. 1706417, 2025. https://doi.org/10.3389/fdata.2025.1706417

[3] P. Xenopoulos, W. R. Freeman, and C. T. Silva, “Analyzing the Differences between Professional and Amateur Esports through Win Probability,” in Proceedings of the ACM Web Conference 2022 (WWW ’22), 2022, pp. 3418–3427. https://doi.org/10.1145/3485447.3512277

[4] S. García-Méndez and F. de Arriba-Pérez, “Explainable e-sports win prediction through machine learning classification in streaming,” arXiv preprint arXiv:2510.19671, 2025.

[5] B. Watono, E. Utami, and D. Ariatmanto, “Stacking Ensemble Learning and Instance Hardness Threshold for Bank Term Deposit Acceptance Classification on Imbalanced Dataset,” Jurnal Teknik Informatika (JUTIF), vol. 5, no. 4, pp. 453–467, 2024. https://doi.org/10.52436/1.jutif.2024.5.4.2022

[6] M. Bansal, A. Goyal, and A. Choudhary, “A comparative analysis of K-Nearest Neighbor, Genetic, Support Vector Machine, Decision Tree, and Long Short Term Memory algorithms in machine learning,” Decision Analytics Journal, vol. 3, Art. no. 100071, 2022. https://doi.org/10.1016/j.dajour.2022.100071